B2 ouc.

gpt.py

```
import math, torch.nn, torch.nn.functional as F.
class GPTConfig:
    pass
    block_size
    vocab_size
    n_layer, n_head, n_embd
    dropout
    bias: True 是否要 bias
```

参数

```
def norm(x):
    F.rms_norm(x, (x.size(-1)))
```

norm_F 替代layer_norm

```
def apply_rotary_emb(x, cos, sin):
    assert x.ndim == 4
    d = x.shape[3]//2
    x1 = x[..., :d]
    x2 = x[..., d:]
    y1 = x1*cos + x2*sin
    y2 = x1*(-sin) + x2*cos
    dtype = torch.cat([y1,y2], 3)
    dtype = y1.x.dtype
    return
```

添加位置信息

旋转编码

```
class CausalSelfAttention(nn.Module):
    def __init__(self, config, layer_idx):
        super
        self.n_embd = config.n_embd.
        self.n_kvhead = config.n_kvhead
        self.n_head = config.n_head
        self.head_dim = self.n_embd // self.n_head.
        self.layer_idx = layer_idx
        assert %
        assert ≤ %
        self.c_q = nn.Linear(n_embd, self.n_head * self.head_dim, bias=False)
        c_k
        c_v
        self.c_proj = nn.Linear(n_embd, n_embd, bias=False)
```

注意力机制

```
def forward(self, x, cos_sin, kv_cache):
    B, T, C = x.size()
    q = self.c_q(x).view(B, T, self.n_head, self.head_dim)
    k
    v
    k = k.view(B, T, self.n_kvhead, self.head_dim)
    v
```

投影矩阵W.B

$\Rightarrow$ 分别 q, k, v

q norm

$\Rightarrow$ 把H和T换

```
    q, k, v = q.transpose(1,2), k.transpose(1,2), v.transpose(1,2)
    q = apply_rotary_emb(q, cos, sin)
    k = apply_rotary_emb(k, cos, sin)
    q = norm(q)
    k = norm(k)
```

① 取出 kv_cache

```
    if kv_cache is None:
        k = torch.cat([k, k], dim=-2)
        v = torch.cat([v, v], dim=-2)
    Tq = q.size(2)
    Tk = k.size(2)
```

② 注意力机制

```
    enable_gqa = self.n_head != self.n_kvhead
    if kv_cache is None or Tq == Tk:
        y = F.scaled_dot_product_attention(q, k, v, is_causal=True, dropout_p=..., enable_gqa=...)
    elif Tq == 1:
        y = F.scaled_dot_product_attention(q, k, v, is_causal=False, dropout_p=..., enable_gqa=...)
    else:
        attn_mask = torch.zeros((Tq, Tk), dtype=torch.bool, device=q.device)
        prefix_len = Tk - Tq
        attn_mask[:, :prefix_len] = True
        attn_mask[:, prefix_len:] = torch.tril(torch.ones((Tq, Tq), dtype=torch.bool, device=q.device))
        y = F.scaled_dot_product_attention(q, k, v, attn_mask=attn_mask, dropout_p=..., enable_gqa=...)
    y = y.transpose(1,2).contiguous().view(...)
    y = self.c_proj(y)
    return
```

'attention 实现'

$q = [B, H, T_q, D]$ $k, v = [B, H_{kv}, T_k, D]$

$\Rightarrow q \cdot k^T = [B, H, T_q, T_k]$

$\Rightarrow \frac{q \cdot k^T}{\sqrt{D}} = [B, H, T_q, T_k]$

$\text{softmax} \Rightarrow [B, H, T_{q_1}, T_k] = \text{prob}$

$\Rightarrow \text{Attn} = \text{prob} \cdot v = [B, H, T_{q_1}, T_k] \cdot [B, H, T_k, D]$

$= [B, H, T_{q_1}, D]$

$\Rightarrow$ contiguous().view() $\Rightarrow [B, T_{q_1}, C]$

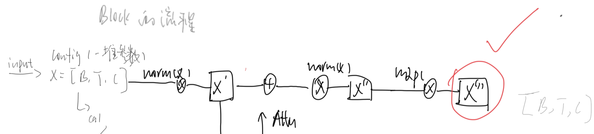
一般是 $T_q = 1$

$\Rightarrow [B, 1, C]$ 一个时间步

class mlp easy

```
class Block(nn.Module):
    def __init__(self, config, layer_idx):
        self.attn = CSA(config, layer_idx)
        self.mlp = MLP(config)
        super().__init__()
    def forward(x):
        x = x + self.attn(norm(x), cos_sin, kv_cache)
        x = x + self.mlp(norm(x))
    return x
```

Block 的实现流程



I

Attn中

X.size 找 B, T, C

找 q:

法找 $[B, T, C] \xrightarrow{W^*} [B, T, n_head \times n_head]$

$\Rightarrow$ view 法 $\rightarrow [B, T, n_head, n_headdim]$

$\Rightarrow$ norm(q) 这怎样把位置信息

找 k

$\Rightarrow [B, T, C] \xrightarrow{W^*} [B, T, n_kvhead \times n_headdim]$

$\Rightarrow$ view 法 $\Rightarrow [B, T, n_kvhead, dim]$

找 v

V3 上.

II 如果 kv_cache 为空 $\Rightarrow$ k, v 更新 把 kv_cache

$q = [B, H, T_q, n_headdim]$

$k = [B, H_{kv}, T_{kc}, n_head_dim]$

$v =$

if kv_cache is None or $T_q = T_{kc}$:

$y = F.sdpa(q, k, v, is_causal = True, enable_...)$

elif $T_q = 1$

else:

Attn_mask = 上图

III 找到 Attn $[B, H,$

$y = y.\text{trans}$